Step 1 — The Typed Shape and the Deterministic Validator — Hints

> Back to the step · Hint 1 · Hint 2 · Solution

- Hint 1 — the two models, and why every field is optional
- Hint 2 — the validator, rule by rule

Hint 1 — the two models, and why every field is optional

Two Pydantic models:

- `LineItem(description: str, quantity: int, unit_price: float)`.
- `Invoice(vendor, invoice_number, line_items, total)`.

Make **every `Invoice` field optional** — `Optional[str] = None`, `List[LineItem] = []`, `Optional[float] = None`. This is the key decision: if a field is required and missing, the model is tempted to invent a value to satisfy the type. Optional fields let a missing value come back as `None`, which your validator then reports honestly as a problem. Plus the standard `config.py` / `exceptions.py` / `utils/logger.py` / `models/llm.py` from every project.

Hint 2 — the validator, rule by rule

`validate_invoice(invoice)` returns a list of problem strings — empty means clean:

```
problems = []
if not invoice.invoice_number: -> "missing invoice number"
if not invoice.vendor:         -> "missing vendor"
if not invoice.line_items:     -> "no line items found"
if invoice.total is None:      -> "missing total"
else:
    computed = sum of (item.quantity * item.unit_price)   # a plain for loop
    if abs(computed - invoice.total) > 0.01:
        -> "total mismatch: line items sum to X, invoice says Y"
return problems
```

Write `compute_subtotal` with a plain `for` loop (not a comprehension). Test it with **hand-made** `Invoice` objects, no model: a good one (no problems), one with `total = 99.0` (mismatch caught), one with `invoice_number = None` (caught). The whole file set is in the Solution.